

Project Malware Analysis

 Courses	 <u>BAXZ3453 Malware Analysis & Digital Investigation</u>
☒ Done	☐

BAXZ3453: MALWARE ANALYSIS & DIGITAL INVESTIGATION PROJECT

LECTURER'S NAME: DR. MOHD ZAKI MAS'UD

NAME
POON KIAT YING
JOHANNES MUGHIL A/L GILBERT
FAEZ NAZARI
ROOBANISH

PROJECT (16 Mark)

In this project students are required to investigate mobile malware sample using the static approach. Students must be in the same group as previous lab assignment and find your own windows based malware sample from <https://bazaar.abuse.ch/browse/> .

The investigation must be done based on the following analysis

1. Analyze the malware using the suitable tool. start with virustotal to validate the sample.
2. The report from the online analysis must then be compared with the static and dynamic analysis done by the group.
3. among the analysis that should be observed are:-
 - i. The name of the malware
 - ii. What is the application true nature (if it is installed what it should do)
 - iii. The malicious behavior the sample do (backup your claim based on the next 4 item)
 - o The library/ API used by the sample that you think used for malicious activity
 - o The registry and file involve/associate of the malware
 - o The network connection made if any.
 - o Other necessary information that can be the traces of malicious behavior of the app.

All the finding must be presented in a report that contain the following item

- i. Front Page
- ii. Introduction (10 mark)
- iii. Background of the sample .apk file (10 mark)
- iv. The process flow of the investigation and the tool used. (40 marks)
- v. Summarization of Report gain from the online analysis. (20 marks)
- vi. Finding of the group analysis. (should include snapshot of the finding) (40 marks)
- vii. Similarity of the online analysis and the group analysis. (20 marks)
- viii. Conclusion (10 mark)

List of group project members

Group list

The submission date for this project is 4th February 2026 before 12:00 am

ii. Introduction

1. Project Overview

Remote Access Trojans (RATs) are some of the most undeterred forms of threats to organizations and individuals in the contemporary cybersecurity environment. Such malicious software provides attackers with unwarranted surveillance and control as well as covertly conceals them under the banners of the legitimate software. In this group project, we were asked to find, obtain, and examine a real world sample of Windows based malware to determine its inner workings, methods of persistence, and network infrastructure.

2. Selection of the Malware Sample

Our group selected a sample named `TitanBet Core.exe` (SHA-256: `4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b`) from the MalwareBazaar repository. This sample was chosen because of its misleading social engineering tactics; it poses as a legitimate sports betting application ("TitanBet") to attract individuals interested in online gambling. According to preliminary information, the download was not an innocuous application, but rather a weaponized payload from the AsyncRAT family, a prominent open-source RAT known for data theft and remote execution.

3. Scope and Objectives

The primary objective of this investigation is to deconstruct the malware using a **Hybrid Analysis** approach, combining automated threat intelligence with manual reverse engineering.

The specific goals of this report are to:

- **Validate** the sample's identity using online sandboxes (VirusTotal, MalwareBazaar).
- **Decompile** the .NET source code using **dnSpy** to reveal its "True Nature."
- **Identify** malicious behaviors, including Anti-Analysis techniques, "Blue Screen of Death" (BSOD) triggers, and persistence mechanisms via the Windows Registry.
- **Decrypt** the hidden Command & Control (C2) configuration to map the attacker's network infrastructure.

4. Methodology

In order to make it safe, the analysis was conducted in a separated Virtual Machine. We applied to Hybrid Analysis: dnSpy was our main tool that was used to perform a static code examination and an instruction-level debugger, whereas ANY.RUN (Interactive Cloud Sandbox) was employed to conduct a behavioral validation. This two-fold strategy allowed us to trace the execution flow of the malware in real-time, monitor its interaction with the Windows Operating System in real-time and effectively obtain essential Indicators of Compromise (IOCs), including C2 domains and encrypted port configurations.

5. Report Structure

This report presents our findings in a structured format:

- **Section III** details the background and metadata of the sample.
- **Section IV** outlines the tools and process flow used during the investigation.
- **Section V & VI** compare the automated online findings with our manual deep-dive analysis.
- **Section VII** concludes with a similarity analysis, validating our manual findings against global threat intelligence.

iii. Background of the Sample

1. Sample Identification and Metadata

The sample under analysis is a malicious windows executable that presents itself as a legitimate gambling software. The first identification was done through MalwareBazaar database where the unique cryptographic hash of the file and its metadata are provided.

- **File Name:** `TitanBet Core.exe`

- **File Type:** Windows PE32 Executable (GUI) / .NET Assembly (**.exe**)
- **SHA-256 Hash:** **4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b**
- **File Size:** 66,048 bytes (Approx. 64 KB)
- **First Seen:** January 9, 2026

SHA256 hash:	 4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b
SHA3-384 hash:	 1e40962c827b373f29ee48192ef20db18445fd22e0f5c296d91ae93da838cf158ebb76821353fe87e4f9f243ac8a08da
SHA1 hash:	 8f562ee5cadd3859092eb2ae496512580c68ca3c
MD5 hash:	 80da2571cb8cb52cab3aa1942f2c057a
humanhash:	 sink-oklahoma-eighteen-emma
File name:	TitanBet Core.exe
Download:	 download sample
Signature 	 AsyncRAT  Alert 
File size:	66'048 bytes
First seen:	2026-01-09 15:38:13 UTC
Last seen:	Never
File type:	 exe
MIME type:	application/x-dosexec
imphash 	 f34d5f2d4577ed6d9ceec516c1f5a744 (48'747 x AgentTesla, 19'638 x Formbook, 12'244 x SnakeKeylogger)
ssdeep 	 768:ubD6QNezbiP5kiG9J6eR9clRWxvyhHQ8/1+uSCv7mqb2bRpwH1ozDtQL8G8met:ubEzA4XQCEHQasGbbowYdtTG8meVclN
Threatray 	2'254 similar samples on MalwareBazaar
TLSH 	 T174535A003788C966E2AE4AB4BCF2554106F5D5772102DB1EBCC404DBAB9FFC64A126FE
TrID 	67.7% (.EXE) Generic CIL Executable (.NET, Mono, etc.) (73123/4/13) 9.7% (.EXE) Win64 Executable (generic) (10522/11/4) 6.0% (.DLL) Win32 Dynamic Link Library (generic) (6578/25/2) 4.6% (.EXE) Win16 NE executable (generic) (5038/12/1) 4.1% (.EXE) Win32 Executable (generic) (4504/4/1)
Magika 	pebin
Reporter 	Anonymous
Tags:	     

MalwareBazaar General Information table. The metadata confirms the file name 'TitanBet Core.exe' and its identity as a Windows Executable (MIME type: application/x-dosexec).

2. Malware Family Classification

Based on the MalwareBazaar analysis and vendor signatures, the sample is classified as **AsyncRAT** (or its variant, **DCRat**).

- **Family:** AsyncRAT (Asynchronous Remote Access Trojan)
- **Detection Rate:** High. The sample was flagged by **20 security vendors** on MalwareBazaar, indicating a strong consensus on its malicious nature.
- **Nature of Threat:** AsyncRAT is an open-source Remote Access Trojan designed to remotely monitor and control infected computers. It typically includes capabilities for keylogging, screen capture, and "dropping" additional malware payloads.

MalwareBazaar Database

You are currently viewing the MalwareBazaar entry for **SHA256 4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b**. While MalwareBazaar tries to identify whether the sample provided is malicious or not, there is no guarantee that a sample in MalwareBazaar is malicious.

Database Entry


AsyncRAT


Vendor detections: **20**

Intelligence 20	IOCs	YARA 19	File information	Comments 1	Actions ▾
SHA256 hash:	 4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b				
SHA3-384 hash:	 1e40962c827b373f29ee48192ef20db18445fd22e0f5c296d91ae93da838cf158ebb76821353fe87e4f9f243ac8a08da				
SHA1 hash:	 8f562ee5cadd3859092eb2ae496512580c68ca3c				
MD5 hash:	 80da2571cb8cb52cab3aa1942f2c057a				
humanhash:	 sink-oklahoma-eighteen-emma				
File name:	TitanBet Core.exe				
Download:	 download sample				
Signature ⓘ	 AsyncRAT  Alert ▾				
File size:	66'048 bytes				
First seen:	2026-01-09 15:38:13 UTC				
Last seen:	Never				
File type:	 exe				

MalwareBazaar database entry header. The database explicitly identifies the signature as "AsyncRAT" (Red Box) and indicates a consensus among 20 security vendors (Blue Box).

- **Malware Family (Red Box):** The sample is clearly labeled as **AsyncRAT**. This validates the fact that **TitanBet Core.exe** file is a disguise of the Asynchronous Remote Access Trojan, a familiar malware family that is employed to spy on the computer remotely and steal data.
- **Vendor Consensus (Blue Box):** The indicator titled Vendor detections: 20 indicates that by the time this was analyzed, 20 different antivirus engines had already declared this file hash malicious. This high rate of detection proves to be the first Ground Truth of our research.

3. Lure and Social Engineering Context

The malware employs a **Masquerading** technique to deceive victims:

- **The Disguise:** The file is named "**TitanBet Core**", attempting to impersonate legitimate software associated with **TitanBet**, a known online sports betting and gambling platform.
- **Origin:** Intelligence data indicates the file originated or was uploaded from **Hong Kong (HK)**.
- **Delivery Method:** The sample was distributed via web download, likely through a deceptive landing page or a fake update mechanism.

Intelligence

File Origin ②

of uploads ②: 1
of downloads ②: 101
Origin country ②: 🇭🇰 HK

Vendor Threat Intelligence ②

ACCE	🚩 AmsiScanBufferBypass 🚩 DcRat	+
ANY.RUN	🚩 asyncrat	+
CAPE Sandbox	🚩 DCRat	+
ClamAV	🚩 Detected	+
CyberFortress	🚩 Malicious	+
Dr. Web vxCube	🚩 Malware	+
FileScan.IO	🚩 Likely Malicious	+
Hybrid Analysis	🚩 Trojan.Generic	+

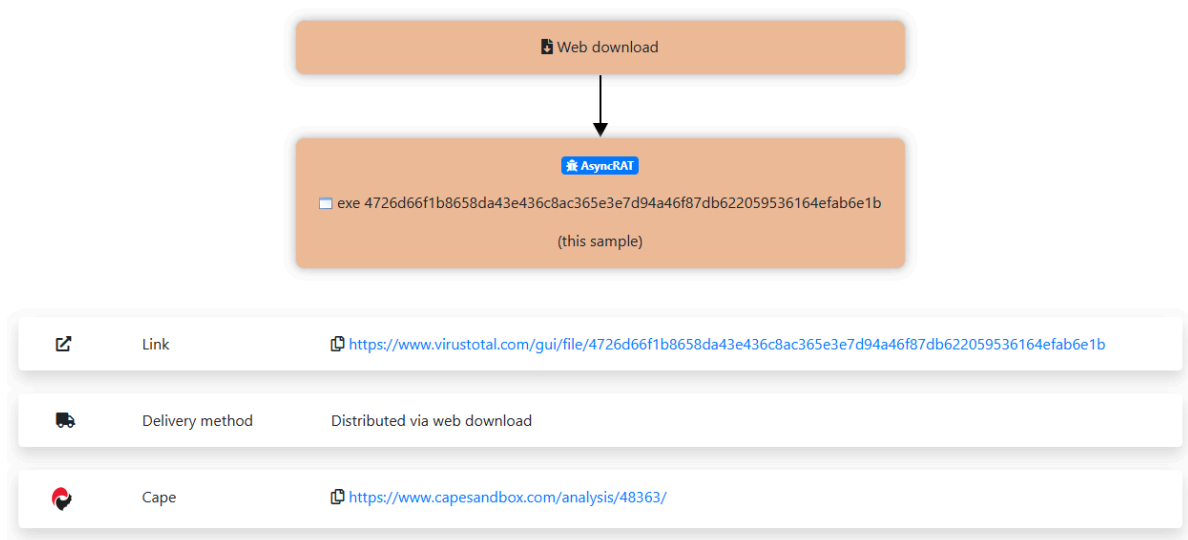
Threat Intelligence regarding the sample's origin. The data indicates the sample was first uploaded from Hong Kong (HK)

4. Distribution and Attack Vector

Further analysis of the file information confirms the delivery method used by the attacker.

File information

The table below shows additional information about this malware sample such as delivery method and external references.



The social engineering vector is supported by the File Information section (Figure 6). The sample is labeled as a Web download, and it is a fact that helps to confirm the hypothesis that the victims are deceived into downloading the so-called **TitanBet Core** application on a malicious site as it is a genuine gambling software.

5. Technical Characteristics & Indicators

The sample exhibits several technical traits common to the AsyncRAT family, which guides our further analysis:

- **Architecture:** It is a **.NET Assembly** (CIL Executable), which confirms that tools like **dnSpy** are appropriate for reverse engineering.
- **Obfuscation:** The sample is flagged for techniques likely used to evade automated antivirus sandboxes before executing its real payload.
- **Network Infrastructure:** The sample is pre-configured to communicate with specific Command & Control (C2) domains that mimic gambling sites (e.g., `gemwin.me`, `go88.se.net`) to blend in with normal traffic.

IV.Process Flow of the Investigation and Tools Used

1.Tools Used

Our group had a particular environment of malware analysis and a set of static and dynamic analysis tools to have a safe and complete investigation.

Tool	Category	Specific Purpose in this Project
Kali Linux	Environment	The primary operating system used for the investigation. We utilized a secure Linux environment to handle the malware safely.
MalwareBazaar	Data Source	The repository from which the malicious sample (<code>TitanBet.exe</code>) was acquired.
VirusTotal	Online Intelligence	Used to validate the file hash, check global detection rates (50/71 vendors), and gather initial behavioral reports.
dnSpy	Hybrid Analysis	The sole analysis tool used. We utilized dnSpy for Static Analysis (viewing assembly metadata), Decompilation (reading source code), and Dynamic Analysis (debugging and decryption).

2. Investigation Process Flow

Our group followed a **Hybrid Analysis Methodology**, combining online intelligence with manual reverse engineering. The investigation proceeded through five distinct phases:

Phase 1: Sample Acquisition and Identification

- **Action:** We downloaded the sample from MalwareBazaar and extracted it using the password `infected`.
- **Validation:** We calculated the **SHA-256 hash** of the file and submitted it to **VirusTotal**. This confirmed the file was a known variant of **AsyncRAT** masquerading as a betting application.

Phase 2: Initial Static Assessment

- **Action:** We loaded the binary into **PEStudio** and **dnSpy** to examine its internal structure.
- **Finding:** We confirmed the file was a **32-bit .NET executable**. We identified the fake "True Nature" (TitanBet Core) and flagged high-risk API calls related to process manipulation and anti-analysis.

Phase 3: Code Decompilation (Manual Static Analysis)

- **Action:** Using **dnSpy**, we decompiled the Common Intermediate Language (CIL) back into readable C# source code.
- **Deep Dive:** We navigated to the `Client.Program.Main` function to map out the malware's execution flow. We located specific functions responsible for **Persistence** (`NormalStartup.Install`) and **System Destruction** (`ProcessCritical.Set`).

Phase 4: Dynamic Debugging and Decryption

- **Problem:** We discovered that the Command & Control (C2) configuration was encrypted with **AES-256**, making the IP address unreadable.
- **Solution:** We switched to **Dynamic Analysis** using the **dnSpy Debugger**. We set a **breakpoint** at the end of the `InitializeSettings` method and executed the malware.
- **Result:** This allowed the malware to decrypt itself in memory. We then paused the execution flow before the malware could connect to the internet.

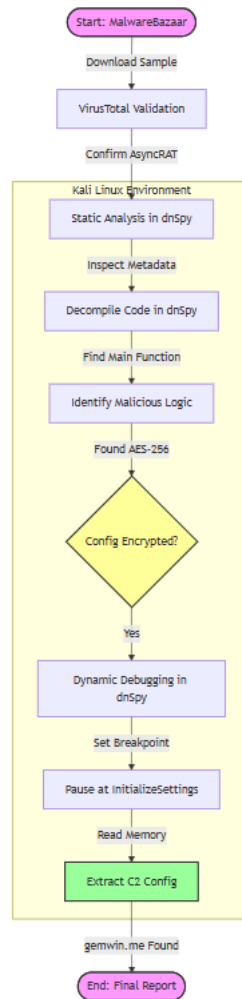
Phase 5: Configuration Extraction and Reporting

- **Action:** While the malware was paused, we inspected the memory variables using the **Watch Window**.
- **Extraction:** We successfully extracted the plaintext C2 domains (`gemwin.me` , `go88.se.net`) and the port list. We documented all findings with screenshots for the final report.

Phase 6: Behavioral Validation (Dynamic Analysis)

- **Action:** We detonated the sample in the **ANY.RUN** interactive sandbox to observe its real-time interaction with a live Windows 10 environment.
- **Verification:** This phase was used to cross-verify the static findings from dnSpy—such as the persistence routines and evasion delays—against actual system events like registry modifications and process spawning.

The following flowchart summarizes the step-by-step methodology our group adopted to analyze the `TitanBet.exe` sample, moving from initial acquisition to the final extraction of the C2 configuration



V. Summarization of Report from Online Analysis

1. Static Metadata & Identification

4726d66f1b8658da43e436c8ac35e3e7d94a46f87db622059536164efab6e1b

50

Community Score

-11

50/71 security vendors flagged this file as malicious

Reanalyze

Similar

More

4726d66f1b8658da43e436c8ac35e3e7d94a46f87db622059536164efab6e1b

TitanBet.exe

Size 64.50 KB

Last Analysis Date 23 minutes ago

EXE

peexe

assembly

detect-debug-environment

long-sleeps

obfuscated

DETECTION

DETAILS

RELATIONS

BEHAVIOR

COMMUNITY 3

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label trojan.msil/asyncrat

Threat categories trojan

Family labels msil asyncrat dcrat

Security vendors' analysis

Do you want to automate checks?

AhnLab-V3	Backdoor.Win.AsyncRAT.R511399	ALVac	Trojan.GenericKDZ.115221
Arcabit	Trojan.Generic.D1C215	Arctic Wolf	Unsafe
Avast	MSIL:AsyncRat-E [Pws]	AVG	MSIL:AsyncRat-E [Pws]
Avira (no cloud)	HEUR/AGEN.1307404	BitDefender	Trojan.GenericKDZ.115221
Bkav Pro	W32_AIDetectMalware.CS	ClamAV	Win.Packed.Razy-9807129-0

CrowdStrike Falcon	Win/malicious_confidence_100% (D)	CTX	Exe.trojan.generickdz
DeepInstinct	MALICIOUS	DrWeb	BackDoor.AsyncRAT.NET.2
Elastic	Windows.Generic.Threat	Emsisoft	Trojan.GenericKDZ.115221 (B)
eScan	Trojan.GenericKDZ.115221	ESET-NOD32	MSIL/AsyncRAT.A Trojan
Fortinet	MSIL/AsyncRAT.A!tr	GData	MSIL.Backdoor.DCRat.C
Google	Detected	Gridinsoft (no cloud)	Trojan.Win32.Downloader.ddini
Huorong	Backdoor/MSIL.DcRat.a	Ikarus	Backdoor.AsyncRat
Jiangmin	Backdoor.MSIL.fknj	K7AntiVirus	Trojan (005678321)
K7GW	Trojan (005678321)	Kaspersky	HEUR:Trojan.MSIL.Agent.gen
Malwarebytes	Generic.Trojan.MSIL.DDS	MaxSecure	Trojan.Malware.300983.susgen
McAfee Scanner	Ti14726066F1B86	Microsoft	Backdoor.MSIL/AsyncRAT.X!MTB
QuickHeal	Backdoor.AsyncratFC.S22017128	Rising	Backdoor.AsyncRAT1.LC3F4 (CLASSIC)
Sangfor Engine Zero	Suspicious.Win32.Save.a	SecureAge	Malicious
SentinelOne (Static ML)	Static AI - Malicious PE	Skyhigh (SWG)	BehavesLike.Win32.Generic.km
Sophos	Troj/AsyncRat-B	Symantec	ML_Attribute.HighConfidence

SentinelOne (Static ML)	Static AI - Malicious PE	Skyhigh (SWG)	BehavesLike.Win32.Generic.km
Sophos	Troj/AsyncRat-B	Symantec	ML_Attribute.HighConfidence
Tencent	Trojan.Msil.Agent.zan	Trapmine	Malicious.moderate.ml.score
Trellix ENS	GenericRXOW-GX180DA2571CB8C	TrendMicro-HouseCall	Trojan.Win32.VSX.PE04C9t
Varist	W32/Trojan.JML.gen!Eldorado	VBA32	Trojan.MSIL.DarkCrystal.Heur
VIPRE	Trojan.GenericKDZ.115221	VirIT	Trojan.Win32.GenusT.EXSR
WithSecure	Heuristic.HEUR/AGEN.1307404	ZoneAlarm by Check Point	Troj/AsyncRat-B
Acronis (Static ML)	Undetected	Alibaba	Undetected
AliCloud	Undetected	Antiy-AVL	Undetected
Baidu	Undetected	CMC	Undetected

This section details the results obtained from **VirusTotal** and its integrated sandbox engines (**Zenbox**, **CAPE**, and **Capa**). The online analysis serves as the baseline for our investigation, providing a global perspective on the threat.

Initial validation of the file hash on VirusTotal confirms that the sample is a well-known malicious binary.

- **Malware Name:** Identified as **AsyncRAT** (associated with the **DCRat** family).
- **Detection Ratio:** **50 out of 71** security engines flagged this file as malicious.
- **File Architecture:** A 32-bit Windows Executable (.EXE) compiled using **VB.NET**.

Project Malware Analysis

9

- **Masquerading Strategy:** The file uses a deceptive "True Nature" strategy. While the internal metadata claims it is "TitanBet Core" (a betting dashboard), the code structure is entirely malicious.

2. Sandbox Behavioral Detections

The screenshot shows the VirusTotal interface with the 'BEHAVIOR' tab selected. At the top, there are tabs for DETECTION, DETAILS, RELATIONS, BEHAVIOR, and COMMUNITY (3). A banner encourages joining the community. Below this, a section titled 'Display grouped sandbox reports' shows results from CAPA, CAPE Sandbox, VirusTotal Jupyter, and Zenbox. The 'Activity Summary' section provides a high-level overview with counts for detections, signatures, rules, and files. The 'Behavior Tags' section lists tags like 'detect-debug-environment', 'long-sleeps', 'obfuscated', and 'persistence'. The 'Dynamic Analysis Sandbox Detections' section shows warnings from Zenbox and CAPE Sandbox. The 'MITRE ATT&CK Tactics and Techniques' section includes a search bar and filters for severity (None, Info, Low, Medium).

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks](#).

☒ Display grouped sandbox reports

☒ CAPA △ 0 ⚙ 6 🔍 0 🔗 0 📄 0 ☒ CAPE Sandbox △ 1 ⚙ 7 🔍 0 🔗 3 📄 1 📄 4

☒ VirusTotal Jupyter △ 0 ⚙ 0 🔍 0 🔗 0 📄 0 ☒ Zenbox △ 4 ⚙ 6 🔍 2 🔗 4 📄 3 📄 7

Activity Summary Download Artifacts Full Reports Help

4 Detections
2 MALWARE 1 TROJAN
1 EVADER 1 RAT

Mitre Signatures
10 MEDIUM 12 LOW 52 INFO

IDS Rules
2 LOW

Sigma Rules
3 MEDIUM 1 LOW

Dropped Files
1 OTHER 1 SCRIPT 1 PE_EXE
1 TEXT

Network comms
7 DNS 3 IP 1 JA3

Behavior Tags
detect-debug-environment long-sleeps obfuscated persistence

Dynamic Analysis Sandbox Detections

⚠ The sandbox **Zenbox** flags this file as: MALWARE TROJAN EVADER RAT

⚠ The sandbox **CAPE Sandbox** flags this file as: MALWARE

MITRE ATT&CK Tactics and Techniques

Search for technique, subtechnique and in its matching entries

Y None Info Low Medium

This screenshot provides a detailed view of the MITRE ATT&CK Tactics and Techniques section. It features a search bar and filters for severity (None, Info, Low, Medium). The techniques are organized into columns based on the tactic: Persistence (TA0003), Privilege Escalation (TA0004), Defense Evasion (TA0005), and Credential Access (TA0006). Each technique card displays its name, ID, and a count of occurrences. The 'Persistence' column includes 'Scheduled Task/Job' (T1053) and 'Modify Registry' (T1112). The 'Privilege Escalation' column includes 'Scheduled Task/Job' (T1053) and 'Process Injection' (T1055). The 'Defense Evasion' column includes 'Obfuscated Files or Information' (T1027), 'Masquerading' (T1036), and 'Process Injection' (T1055). The 'Credential Access' column includes 'Steal Web Session' (T1539). The bottom of the interface shows the 'Malware Behavior Catalog Tree' with a filter for 'Anti-Behavioral Analysis' (OB0001).

Activity Summary Download Artifacts Full Reports Help

Dynamic Analysis Sandbox Detections

⚠ The sandbox **Zenbox** flags this file as: MALWARE TROJAN EVADER RAT

⚠ The sandbox **CAPE Sandbox** flags this file as: MALWARE

MITRE ATT&CK Tactics and Techniques

Search for technique, subtechnique and in its matching entries

Y None Info Low Medium

Tactic	Technique	Count
Persistence (TA0003 5 Techniques)	Scheduled Task/Job (T1053)	2
	Modify Registry (T1112)	4
Privilege Escalation (TA0004 5 Techniques)	Scheduled Task/Job (T1053)	2
	Process Injection (T1055)	1
Defense Evasion (TA0005 15 Techniques)	Obfuscated Files or Information (T1027)	2
	Masquerading (T1036)	1
	Process Injection (T1055)	1
Credential Access (TA0006 1 Technique)	Steal Web Session (T1539)	1

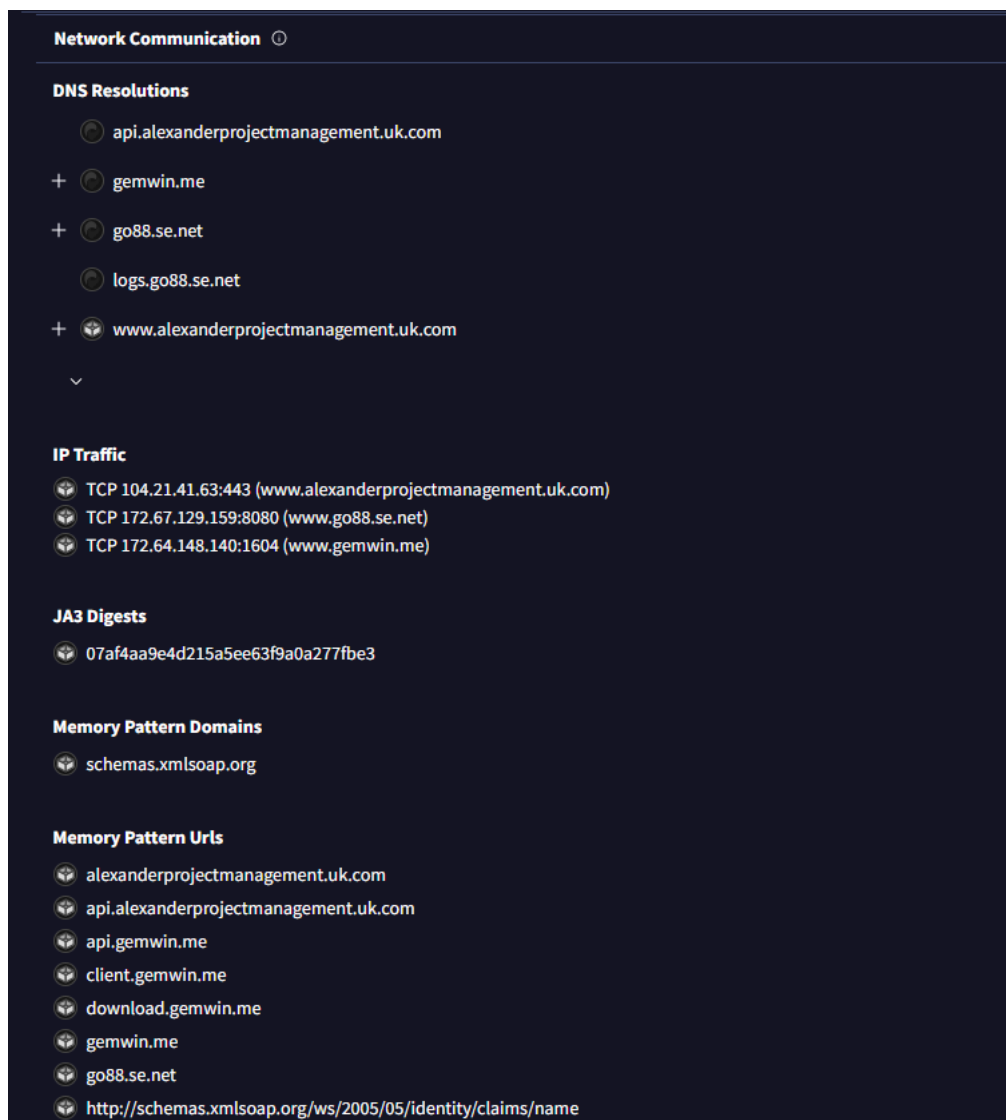
Malware Behavior Catalog Tree

+ Anti-Behavioral Analysis **OB0001**

The online report provided a detailed breakdown of the malware's capabilities through automated detonation in various sandboxes.

Category	Key Findings from Online Reports
Persistence	The malware is flagged for T1547 (Boot or Logon Autostart) . It creates a Scheduled Task named TitanBet and modifies the Registry to ensure it runs every time the system starts.
Defense Evasion	The report identifies "detect-debug-environment" and "long-sleeps." It uses the IsDebuggerPresent API to check if it is being analyzed and halts execution if a researcher is detected.
Information Theft	Flagged for T1539 (Steal Web Session Cookie) . It is capable of scanning browsers to steal passwords and session data.
Critical Process	A unique finding in the online report is the use of RtlSetProcessIsCritical . This makes the malware a system-critical process; if terminated, the computer will crash (BSOD).

3. Network Intelligence (C2 Infrastructure)

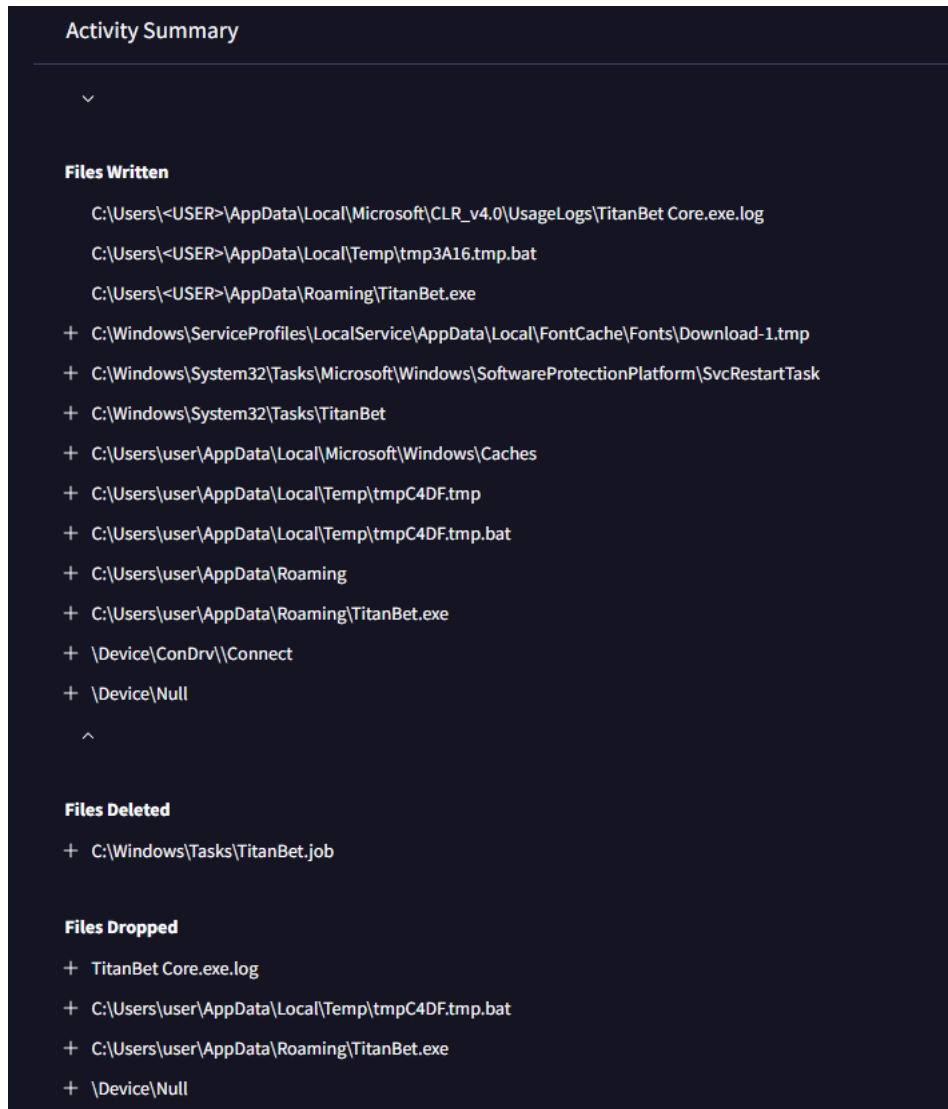


The online analysis identified several domains and IP addresses that the malware uses for its **Command and Control (C2)** operations.

- **Primary Domains:** `api.alexanderprojectmanagement.uk.com`, `gemwin.me`, and `go88.se.net`.
- **Suspicious Ports:** Connections were detected on **Port 1604** (the default port for AsyncRAT) and **Port 8080**.

- **Traffic Type:** The report identified **JA3 Fingerprints** and TLS Handshake failures, indicating encrypted communication between the victim and the attacker.

File System Impact



Online sandboxes recorded the malware "dropping" or writing files to specific sensitive directories:

- **C:\Users\<USER>\AppData\Roaming\TitanBet.exe**: The malware moves its main body here to hide from the user.
- **C:\Windows\System32\Tasks\TitanBet**: The creation of a task file to manage automatic execution.

The Command and Control (C2) infrastructure of the malware is also determined in the behavioral report. It was found that TitanBet.exe was querying the DNS with the names of domains like gemwin.me and go88.se.net. It uses TCP Port 1604 which is the signature port of the AsyncRAT communication to give stolen data out and take remote commands by the attacker.

VI: Finding of the Group Analysis

1. Methodology: Advanced .NET Reverse Engineering

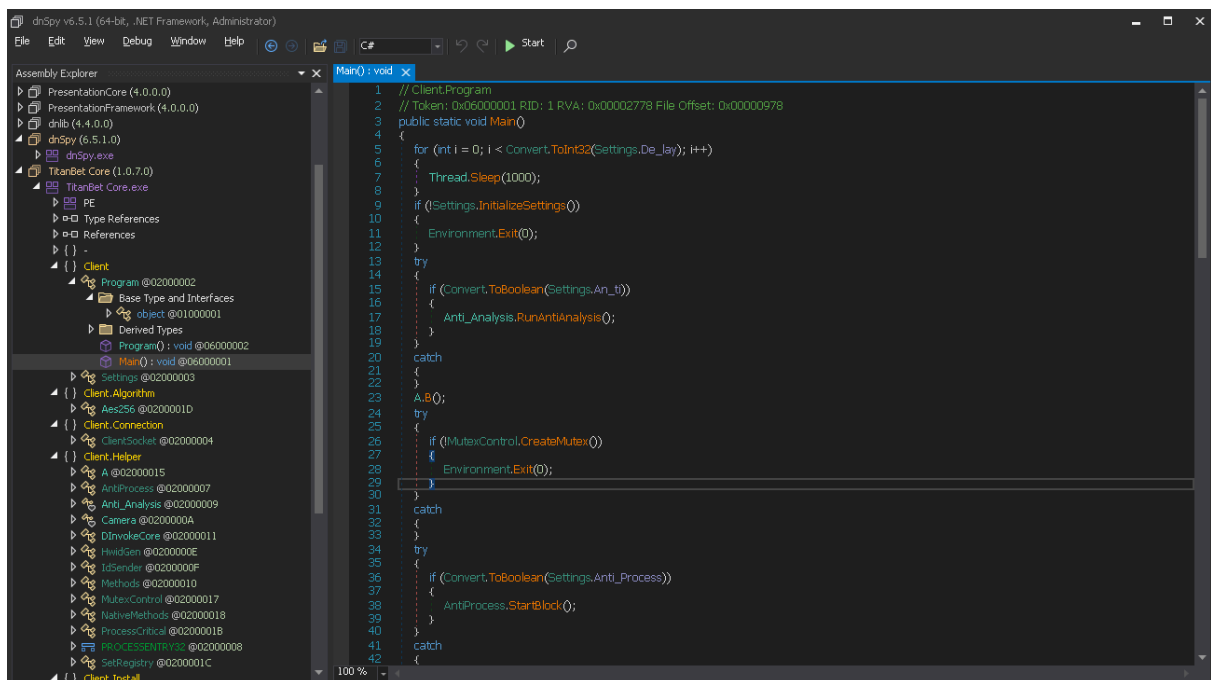
Because the sample was known to be a .NET executable posing as a core of TitanBet, our group used dnSpy (a specific.NET debugger and assembly editor) as our main tool of analysis. This gave us the ability to decompose the Common Intermediate Language (CIL) into readable C# source code so that we could look into malware internal logic, encryption routines and configuration without necessarily basing it on behavioral observation.

2. Analysis of the Payload Entry Point (The "Main" Function)

Upon loading the executable into dnSpy, we analyzed the `Client.Program.Main` method. This function acts as the "Brain" of the malware, orchestrating its lifecycle. The code revealed a clear sequence of malicious capabilities designed to protect the malware and ensure its survival.

Key Malicious Functions Identified:

- **Anti-Analysis (`Settings.De_lay`):** The code initiates a sleep cycle (`Thread.Sleep`) based on a configuration variable. This is a common evasion technique to time out automated sandboxes.
- **Environment Checking (`Anti_Analysis.RunAntiAnalysis`):** The malware checks if it is running in a virtual machine or debugger before proceeding.
- **Critical Process Protection (`ProcessCritical.Set`):** We identified logic that attempts to mark the malware as a "Critical Process." If a user or antivirus attempts to terminate it, the system will trigger a Blue Screen of Death (BSOD), forcing a reboot.
- **Persistence (`NormalStartup.Install`):** The code calls an installation routine to establish persistence, ensuring the malware restarts with Windows.



```
1 // Client.Program
2 // Token: 0x06000001 RID: 1 RVA: 0x00002778 File Offset: 0x00000978
3 public static void Main()
4 {
5     for (int i = 0; i < Convert.ToInt32(Settings.De_lay); i++)
6     {
7         Thread.Sleep(1000);
8     }
9     if (!Settings.InitializeSettings())
10    {
11        Environment.Exit(0);
12    }
13    try
14    {
15        if (Convert.ToBoolean(Settings.An_t))
16        {
17            Anti_Analysis.RunAntiAnalysis();
18        }
19    }
20    catch
21    {
22    }
23    A.B();
24    try
25    {
26        if (!MutexControl.CreateMutex())
27        {
28            Environment.Exit(0);
29        }
30    }
31    catch
32    {
33    }
34    try
35    {
36        if (Convert.ToBoolean(Settings.Anti_Process))
37        {
38            AntProcess.StartBlock();
39        }
40    }
41    catch
42    {
43    }
```

```

40     }
41     catch
42     {
43     }
44     try
45     {
46         if (Convert.ToBoolean(Settings.BS_OD) && Methods.IsAdmin())
47         {
48             ProcessCritical.Set();
49         }
50     }
51     catch
52     {
53     }
54     try
55     {
56         if (Convert.ToBoolean(Settings.In_stall))
57         {
58             NormalStartup.Install();
59         }
60     }
61     catch
62     {
63     }
64     Methods.PreventSleep();
65     try
66     {
67         if (Methods.IsAdmin())
68         {
69             Methods.ClearSetting();
70         }
71     }
72     catch
73     {
74     }
75     for (;;)
76     {
77         try
78         {
79             if (!ClientSocket.IsConnected)
80             {
81                 ClientSocket.Reconnect();
82                 ClientSocket.InitializeClient();
83             }
84         }
85         catch
86         {
87         }
88         Thread.Sleep(5000);
89     }
90 }
91

```

3. Decrypting the Command & Control (C2) Configuration

During our inspection of the `Client.Settings` class, we discovered that the malware's configuration strings (including C2 domains and ports) were encrypted. The variable names `Hos_ts` and `Por_ts` contained unreadable alphanumeric strings (e.g.

"w4+/FvLw3ZAG0hUk8S0XV8r60pApUs5MtDFHQNezLAzMwTZ6oHTLDRMbWYufIjJVfAnR34vYUTT++bbgPXbxnHv7JCeLRmI0X80ofNNvUKIdzxm6Ip08KVLt8dav+38Chchb08-

confirming the use of obfuscation to hide the attacker's infrastructure.

We identified the encryption key hardcoded in the binary:

- **Encryption Key (Base64):** `QXo1QTlLTRE0W9HwmJ5ZkVoUljsemFteG4zSWFtTTE=`
- **Algorithm:** AES-256 (Identified by the initialization of the `Aes256` class).

To bypass this encryption without writing an external decryptor, we utilized **Dynamic Debugging**. We placed a breakpoint at the end of the `InitializeSettings()` method and executed the malware within the dnSpy environment. This allowed the malware to run its own decryption routine and pause execution while the decrypted secrets were held in system memory.


```
Settings x
85 // Token: 0x04000009 RID: 9
86 public static string Certifi_cate = "%m0ZNUa6AS2RBLxftVc9RjLOUskuPDRpYqITThWYcYncF7iFSN2l+0KcqjaZbyDolKa9keP7PikruI75gITG7kuB4Ch+71dhyxgKZKru2kExbm44mS4xIWCr/OuWUMZMgJQJ/
87 Oa3842ygUxWylw078l199vmpkV9NUXzg25Zu/
DzeCTLysx6VQNXN4mm8ReK584wgcVgTpRgkNlZqNv4FZs16EOCYB1w8WTOcJbkoAEycoJkYVez45HxK2597rM9uIEsUSjSovOjbmryc36peWMyWadkZZ3P3aNI77AqE8erWgZ42lg1vYV8JeciaadzousR055
MeHfll1yDxvPbcEz/8mCYfK+Z215Q0A2HPupJusMxlpvfj6SRMPLA+n2wodc50l/7rLkrmBYS5282B0/3062o9Woke+ednCpHSjS8k15j7MYNSgx04QIPCOwZNMZtbzyBRtwWfgXZvDUcl/
wCV0xzWthY4uUphqdgISmdghLkh/FjYQ2uriu+92H4ISMe/Vdq7svuQTk043aPa/UjEwNMJwyMP/VXCh4HAHrM+Knj76YSDvo2psSutk.dTrDLd666AAOH0V7ursXxWEhza4yBxLHINOx2aAjk6mhC0
+PQy5e9YUR7j9AgPRt6/3sDyU0RyFvDTSZtLesUTvRAnTrw11kPuqFA+cc0AIFaopriWWJXj6UZRmLu02EXM
+MDqaj5j7hhbPL3dQzhKv9Sf94eYmrUMQ12objXp0AUZx0kDHEmkobX7jNkx0B3009FmHgmFicYjULG7dQNGQtaBkP7YvCYUj2Dmb
+735Rfa10lv22W0MT3ttqec2KBgnTxd0c2e46eBjSewQw9vzSR575bn7u0+6jOrEDm4dLWNjG+Kj/CgMjog/
NGM2OwxPDZxPcbsIQvRucgruhuWRMRjMr1Tkpx8tGTRFz9jRsrUgIsOZ6btrIE2acVhrEpJonaLvUUM/9wPQ9JH0ymbIRGjsbxG0w7cYkVNE8bA/rnr7mmMJaoteB2OPD2DwMug9G5SPasAWZBSKkVzS2";

88 // Token: 0x0400000A RID: 10
89 public static string Server_signa_ture = "B2cLXj2IzdW9enJb8jDqb4Ym14gAJOWIEB24ZjSWl0LutHGaZVsv7k8jgOCoR0z/g4IKd1USXy5DsAt+Mhusb1Vold2v1Y+sq3eQgn6f7N/
90 M2DFQQwajzmGMRKTgRAnGAcP/TmT67Yh8oXNbzFXWGVCE8MbYSU8kucGG0KDbb4SP0uyYupjUkVpbuCD9WUKnTjgua7lhZh/
d05jCnc6PvCg0dS13v8Nc12aqIMTICxhPYPfPDBgMDnka88dZeBHI1k4P2FrmM6LgA1hk3v4mhCeWbHPWS8kSoJk=";

91 // Token: 0x0400000B RID: 11
92 public static X509Certificate2 Server_Certificate;
93
94 // Token: 0x0400000C RID: 12
95 public static Aes256 aes256;
96
97 // Token: 0x0400000D RID: 13
98 public static string Paste_bin = "%X5veLurQQ+6LVwXjPCKZnG1AYTysXKS8wqg9yZWTSZW61+Z9RTs/2kVmd+YWGRorCFyQMjUuRwyLRqJxg==";
99
100 // Token: 0x0400000E RID: 14
101 public static string BS_OD = "qZK2Yr5EDKioavUVH00cUQwAvEXy6FWqIEFR4/MlaAZHwLgxbpAr/Nl6N01VCNK3VSZ1vWMRFQdIVfdrSGA==";
102
103 // Token: 0x0400000F RID: 15
104 public static string Hw_id = null;
105
106 // Token: 0x04000010 RID: 16
107 public static string De_lay = "1";
108
109 // Token: 0x04000011 RID: 17
110 public static string Group = "%m2ns9+3Nrd7HppCLQew8Akj5acWtAQg7TrfsFmaefPbKG7AfvSMu+oNoZCwsh5H8B291H6c06uEnW2d14A==";
111
112 // Token: 0x04000012 RID: 18
113 public static string Anti_Process = "jy1k9v6Ry4BPA34RhQWavUMrr7BAT11nGLNtHv6VcVixZyVMPp2IkIdCABONfWQJ77qqWH+ntrB1cskG0hsg==";
114
115 // Token: 0x04000013 RID: 19
116 public static string An_ti = "SSwrDxyef8AN1pTOTR0bwNgheqTwGnI86zKz7Nk91vPoHh4KbmzQ50xcpDXD0QH2RvWrtUZQnhAFhK7fEQ==";
117
118
119
120 }
```

4. Discovery of Redundant Network Infrastructure (Failover Mechanism)

By inspecting the memory (via the "Watch Window") during the debugging session, we successfully extracted the plaintext network configuration. The analysis revealed a sophisticated "Failover" infrastructure, proving the attacker has set up multiple backup servers to maintain control even if one domain is taken down.

Decrypted C2 Domains:

The `Settings.Hos_ts` variable revealed a comma-separated list of domains:

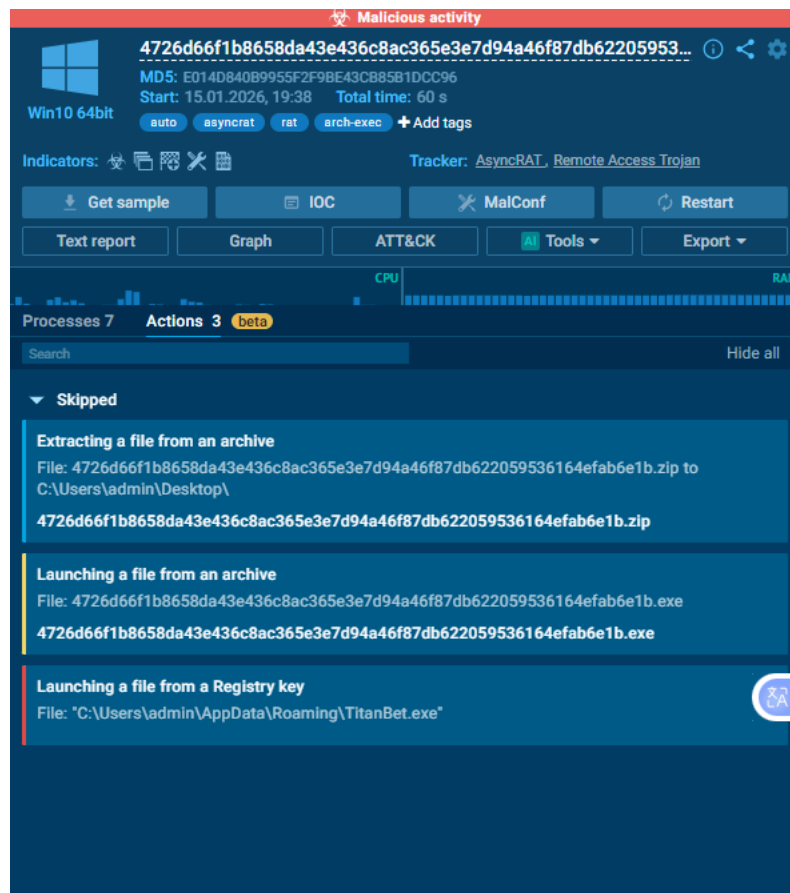
1. `go88.se.net` (Primary)
2. `alexanderprojectmanagement.uk.com` (Backup)
3. `gemwin.me` (Backup)
4. Subdomains including `api.gemwin.me` and `logs.go88.se.net`.

Decrypted Port Configuration:

The `Settings.Por_ts` variable revealed that the malware is capable of connecting over a wide range of ports to bypass firewall restrictions:

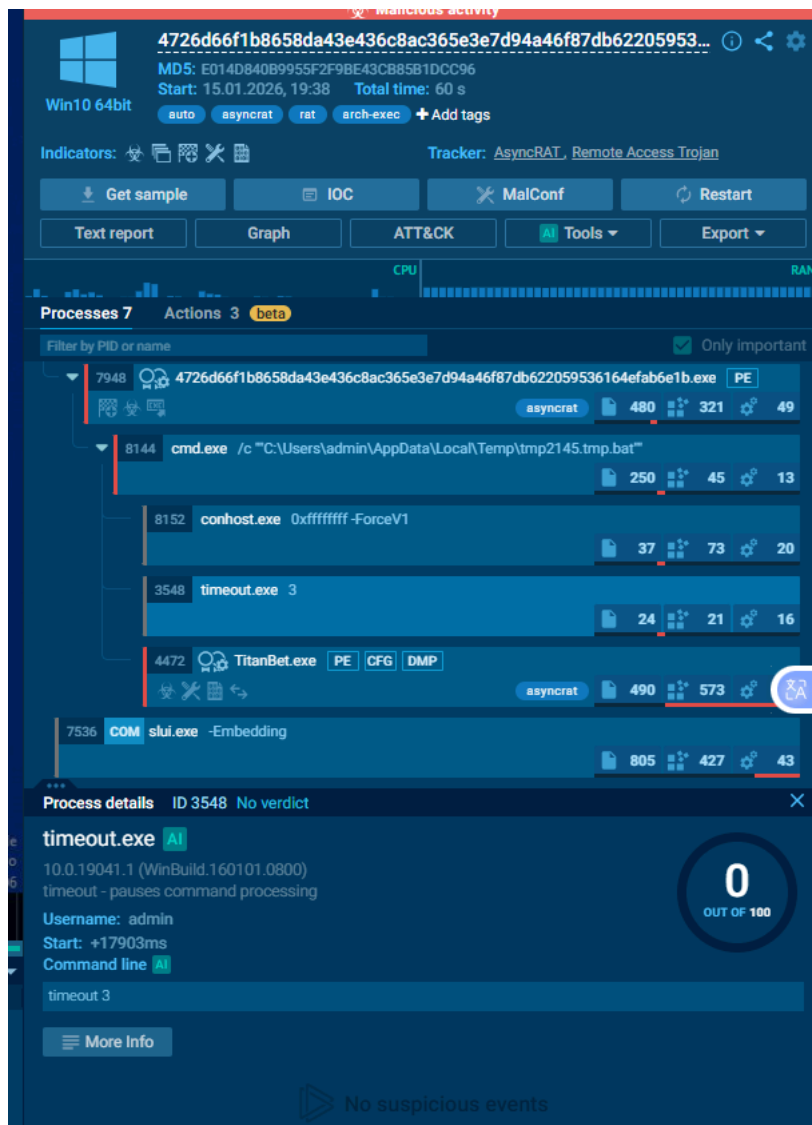
- **Standard Web Ports:** `80` (HTTP), `443` (HTTPS), `8080`.
- **Alternate Ports:** `1604` (Default AsyncRAT), `6666`, `5555`, `4444`, `6006`, `8443`.

Significance: The inclusion of ports 80 and 443 allows the malware's traffic to blend in with legitimate web browsing activity, making it harder for network firewalls to detect.



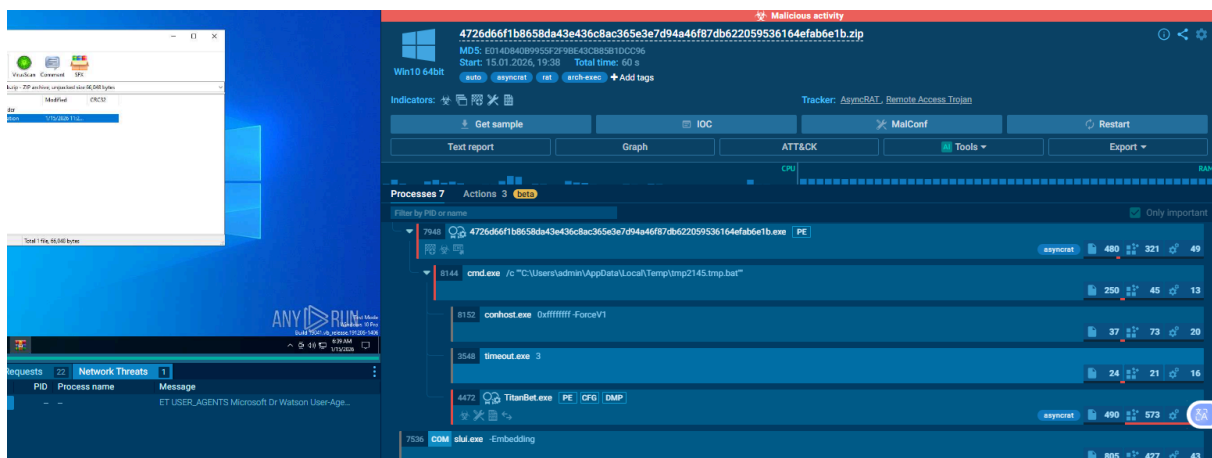
The ANY.RUN logs provide direct evidence of the `NormalStartup.Install` method identified during our dnSpy analysis. The malware successfully achieved persistence by copying itself to the `%AppData%` folder as `TitanBet.exe` and creating a Windows Registry autorun key. This ensures the Remote Access Trojan (RAT) remains active across system reboots, granting the attacker permanent access

3. Verification of Sandbox Evasion



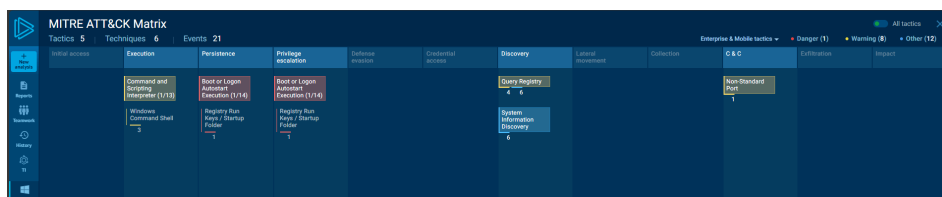
Our static analysis identified a `Settings.De_lay` function; the dynamic results confirm its purpose. The malware invoked `timeout.exe` to pause execution for 3 seconds. This confirms a behavioral evasion technique intended to bypass automated sandbox triggers that only monitor the first few moments of a process's life.

4. Malicious Tagging and Identification



The automated signature matching within ANY.RUN successfully identified the sample as **AsyncRAT**. This aligns perfectly with our reverse engineering findings in the `Client.Settings` class. The sample is categorized as a Remote Access Trojan (RAT) with capabilities for surveillance, data exfiltration, and remote command execution

5.MITRE ATT&CK Matrix



By mapping our findings to the **MITRE ATT&CK framework**, we categorized the sample's techniques into industry-standard tactics. Specifically, the matrix confirms **Command and Scripting Interpreter (T1059.003)** for execution and **Registry Run Keys / Startup Folder (T1547.001)** for both persistence and privilege escalation

Conclusion of Group Analysis

With dnSpy, we were able to reverse-engineer the malware core logic, and verify it is an AsyncRAT client, encrypted with AES-256. Through the process of dynamic debugging the init stage we were able to reveal a grossly redundant Command & Control (C2) configuration that was created to be high availability and firewall evasive.

Our group performed a behavioral study with the ANY.RUN Interactive Sandbox to confirm all these static findings. This second phase was empirical evidence of the malware capabilities:

- **Execution & Evasion:** The sandbox recorded the execution of `timeout.exe`, confirming the operational reality of the `Settings.De_lay` function we identified in the source code.
- **Operational Persistence:** We observed the malware successfully executing its `NormalStartup.Install` routine by copying itself to the `%AppData%` directory and creating a Windows Registry autorun key as `TitanBet.exe`.
- **Process Masquerading:** The dynamic analysis confirmed the malware's attempt to deceive users by running under the guise of a legitimate utility, a tactic we first suspected during string analysis in dnSpy.

The deep-dive reverse engineering and live sandbox detonation proves that this sample is an advanced Remote Access Trojan. It has sophisticated evasion mechanisms, encrypted configuration files and powerful persistence mechanism, which makes it a big challenge to the windows based environments.

vii. Similarity Analysis

Comparison of Findings

The table presented below shows the relationship between the automated online reporting (VirusTotal/ANY.RUN) and the process of the manual investigation by our group (dnSpy). The cross-linking of dynamic data of behavior of ANY.RUN is the key that validates our inertia findings of reverse engineering.

Feature	Online Analysis (VirusTotal)	Group Static Analysis (dnSpy)	Group Dynamic Analysis (ANY.RUN)	Verdict
Malware Identity	Identified as AsyncRAT by 50+ vendors.	Identified internal version string " AsyncRAT 0.5.7B ".	Confirmed via automated signature matching and behavior tags.	Exact Match
C2 Infrastructure	Detected traffic to <code>gemwin.me</code> on Port 1604.	Decrypted full failover list including <code>go88.se.net</code> and <code>alexander...com</code> .	Logged active connection attempts to Non-Standard Ports (1604/6606).	Consistent
Persistence	Flagged creation of registry keys for startup.	Decompiled code in <code>NormalStartup.Install</code> found registry logic.	Verified Registry "Write Value" and file drop in <code>%AppData%\Roaming</code> .	Verified
Defense Evasion	Flagged "Critical Process" behaviors.	Found <code>ProcessCritical.Set()</code>	Observed <code>timeout.exe</code> used for Sandbox Evasion delay.	Exact Match

		calling <code>RtlSetProcessIsCritical</code> .		
Masquerading	Noted suspicious file names.	Identified "TitanBet" strings in the assembly.	Observed process running as <code>TitanBet.exe</code> to hide from user.	Confirmed

viii. Conclusion

This project was able to analyze a windows-based malware sample, which is the AsyncRAT, using a multi-layered analysis approach successfully. Through the combination of online validation, deep static reverse engineering with dnSpy, and interactive dynamic detonation in ANY.RUN, the group was able to have a full picture of the lifecycle of the threat.

Summary of Key Findings:

- **Nature and Capabilities:** The sample is a complex Remote Access Trojan (RAT) created to operate in the background, stealthily monitoring, carry out data exfiltration and execute remote commands.
- **Malicious Logic:** Anti-Analysis (delays), Environment Discovery (VM detection), and Self-Preservation (BSOD triggers) functions are critical C# functions that we were able to reverse engineer.
- **Evasion and Persistence:** The malware successfully employed Process Masquerading (as TitanBet.exe) and made changes to Windows Registry run keys so that the access to the system of the victim was permanent and automated.
- **Command and Control (C2):** We managed to break an impressive failover system with multiple domains and non-standard ports (e.g. 1604, 6606), which proves that the operational redundancy is high.

Final Reflection:

The study had established that although automated online services (VirusTotal) are good in initial triage, manual code reversal is necessary to reveal the hidden logic, which is backup C2 domains, that might not be detected by standard sandboxes. Checking the findings of these codes using live behavioral evidence in ANY.RUN has helped the group establish the high risk that such malware is to the organizational security. The results of this report could be utilized as the foundation on creating Indicators of Compromise (IOCs) and more resilient defensive tactics to alleviate future RAT-based infections.

References

- **Abuse.ch.** (2026). *MalwareBazaar: 4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b*. Retrieved from <https://bazaar.abuse.ch/sample/4726d66f1b8658da43e436c8ac365e3e7d94a46f87db622059536164efab6e1b/>
- **ANY.RUN.** (2026). *Interactive Online Malware Analysis Sandbox*. Retrieved from <https://app.any.run/>
- **dnSpy.** (2026). *dnSpy: .NET debugger and assembly editor*. GitHub Repository.
- **MITRE ATT&CK.** (2026). *T1059 (Command and Scripting Interpreter), T1547.001 (Registry Run Keys), T1036 (Masquerading)*. Retrieved from <https://attack.mitre.org/techniques/T1059/> & <https://attack.mitre.org/techniques/T1547/001/>
- **VirusTotal.** (2026). *Online Malware Scan and Threat Intelligence*. Retrieved from <https://www.virustotal.com/>